



POSTMORTEM → REGRESSION GUARD

Never Again

Every postmortem becomes a guard your code can't forget.

Built with IBM Bob 2.0
github.com/harshgitty58/never-again-guard

● **Team Kira**

Harsh Dhavane · TY Computer Engineering, VIT Pune



Postmortem action items drift, and the same bug comes back.

Items live in a doc

"Add a test", "check similar code", "update the runbook". Tracked as tickets, deprioritized in the next sprint, quietly dropped.

"We added a test"

Nothing proves that test would have failed on the broken code. A test that always passes guards nothing.

Same mistake, other files

The root-cause pattern is almost always copy-pasted elsewhere, and nobody goes looking.

"A merged pull request proves that code changed. It does not prove that the postmortem action controls the failure."

OneUptime, verifying postmortem actions (2026)

far more likely

recurrence when action items are incomplete

Google SRE Workbook

45%

repeat-incident rate one SRE team measured before enforcing follow-ups

Tanimawo, DEV Community

Prove, don't claim. Bob generates; a deterministic verifier judges.

@ inc-102-bug

EXPECT FAIL

```
$ vitest run regression.test.js
X FAIL failedOn: assertion
  expected 10 to be less than or equal to 4
exit 1 · 1.1 s
```

@ inc-102-fix

EXPECT PASS

```
$ vitest run regression.test.js
✓ PASS all assertions hold

exit 0 · 3.2 s
```

Same test, two throwaway git worktrees. A crash or import error on the bug commit doesn't count. Anti-cheat rejects skipped, assertion-free or self-mocking tests.

One postmortem in, four verified outputs out.

1 · Regression test

Fails on an assertion at the bug commit and passes at the fix commit. Proven both ways.

2 · Variant rule

A Semgrep rule generalized from the root cause. It must match the bug and not the fix, then it scans the whole app.

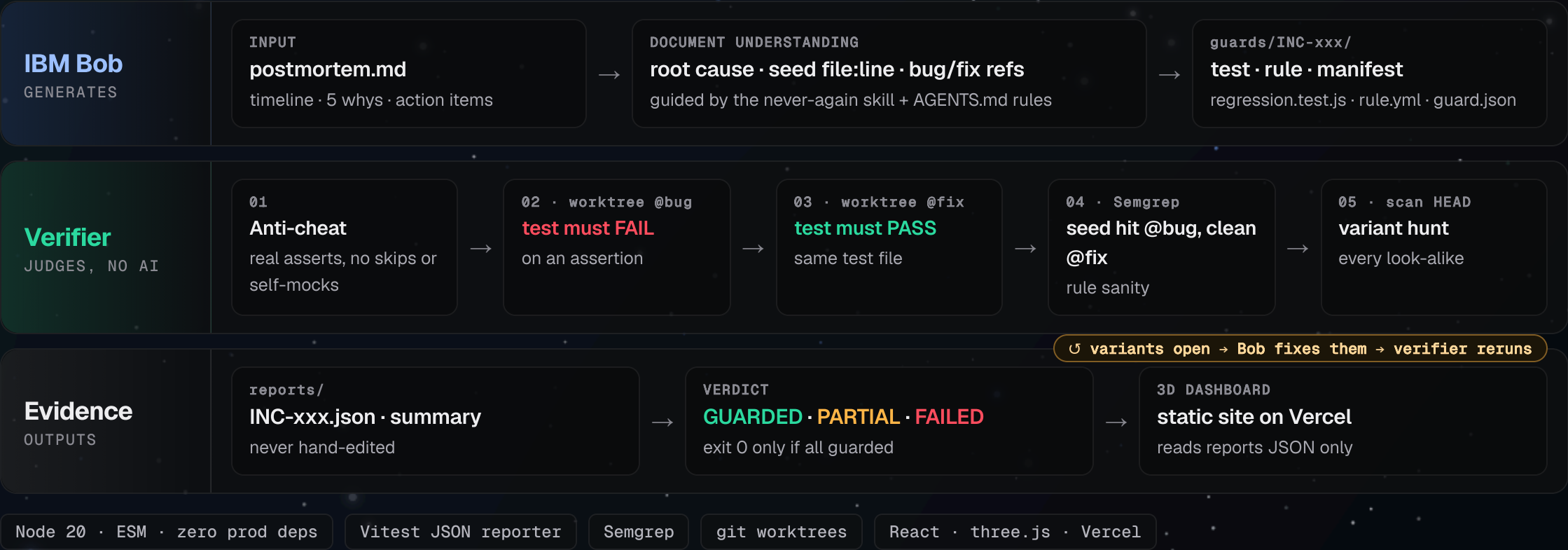
3 · Variant fixes

Bob fixes every variant the rule finds. The verifier reruns until none are open.

4 · Action-item audit

Every item is marked guarded / partial / unguarded / process-only, with file evidence. No invented code for runbooks.

Two lanes, one rule: whatever writes the guard never grades it.



A 4-minute gateway blip became a 47-minute outage. Here's how it never happens again.

SEV-1 · 14 Sep · 47 min

Retry storm took down payments

~48,000 req/min from zero-delay retries

API key blacklisted for 43 min

1,847 failed checkouts

₹28.4 lakh estimated lost GMV

root cause

withRetry(fn) with no options: infinite attempts, 0 ms backoff

Bob reads the postmortem

seed payment.js:19, fix ref, 5 action items

Bob writes the test

a fake gateway that fails, then caps itself at 10 calls; assert calls ≤ 4

@ inc-102-bug: ✗ fails on assertion

expected 10 ≤ 4 in 1.1 s: it catches the real bug

@ inc-102-fix: ✓ passes

bounded retries with backoff, 3.2 s

Rule withRetry(\$F) finds 2 more

clients/inventory.js:13 · clients/sms.js:13, the same outage waiting to happen

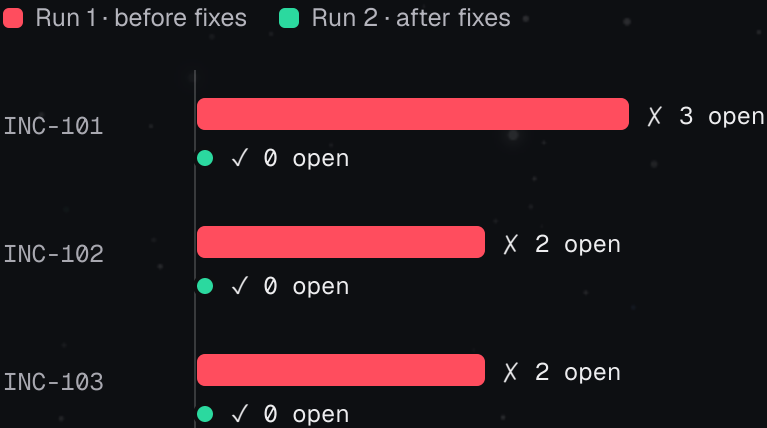
Bob fixes both, verifier reruns: GUARDED

0 open variants · 4 of 4 code action items with evidence

All three incidents proven both ways. 7 look-alike bugs closed.

	ANTI-CHEAT	TEST @BUG	TEST @FIX	RULE @BUG	RULE @FIX	VERDICT
INC-101 time zone	✓	✗ fails 0.9 s	✓ passes 1.3 s	✓ hit	✓ clean	GUARDED
INC-102 retry storm	✓	✗ fails 1.1 s	✓ passes 3.2 s	✓ hit	✓ clean	GUARDED
INC-103 ₹NaN checkout	✓	✗ fails 1.0 s	✓ passes 1.0 s	✓ hit	✓ clean	GUARDED

Open variants per incident



INC-101 rule · raw Date getters (UTC, not IST)

loyalty.js:17 loyalty.js:18
reports/daily.js:18

INC-102 rule · withRetry(\$F) without options

clients/inventory.js:13 clients/sms.js:13

INC-103 rule · arithmetic on raw .price

invoice.js:16 pricing/cartTotal.js:14

Follow-ups go from "we think it's done" to evidence.

Postmortem action items, before and after (15 items across 3 incidents)

Run 1

✓ 6 guarded

✗ 5 unguarded

4 process-only

Run 2

✓ 11 guarded · 0 unguarded

4 process-only

Process-only items (runbooks, on-call, CI policy) are labeled honestly, never given invented code evidence.

3/3

incidents guarded

7 → 0

hidden variants closed

<1 min

to re-prove everything

1 task

IBM Bob, 39.26 Bobcoins

WITHOUT NEVER AGAIN

- ✗ "Added a test" · never run against the bug
- ✗ Look-alike files unchecked
- ✗ Runbook item marked "done"

WITH NEVER AGAIN

- ✓ Test proven: fails on the bug, passes on the fix
- ✓ 7 look-alikes found and closed
- ✓ Process-only items labeled as process-only

On-call / SRE

know the fix is real before closing the incident

Engineering managers

audit follow-ups by evidence, not ticket status

Platform teams

turn every incident into a reusable rule

From a hackathon demo to a gate every incident passes through.

● NOW · SHIPPED

- Verifier CLI with anti-cheat and worktree proofs
- 3 incidents GUARDED, 7 variants closed
- never-again skill for IBM Bob
- 3D dashboard on Vercel

● NEXT · 30 DAYS

- **GitHub Action**: guard every merged postmortem PR
- **CI gate**: incident rules block PRs that reintroduce the pattern
- **Guard-decay check**: nightly re-proof that the test still fails on the bug
- Import from **incident.io**, **Jira**, **PagerDuty**

● LATER · WITH IBM

- **watsonx.ai** (Granite) extracts postmortem fields for teams without Bob in the IDE
- **watsonx Orchestrate** chases process-only items: owners, due dates, reminders
- **Fault-injection drills** to prove runtime controls, not just code
- Org metric: **repeat-incident rate**, trending to zero

Every postmortem, proven. **Never again.** · github.com/harshgitty58/never-again-guard